

The Effects of Covid-19 Analysis Python Project

Αναστασία Σουλελέ 1084612

May 15, 2023

Contents

1	Introduction	2
2	Python Coding	3
2.1	Libraries/Download the file from the website	3
2.2	GUI creation	4
2.3	Graphs	7
2.3.1	Συνολική παρουσίαση του τζίρου (στήλη value) ανά μήνα (στις αντίστοιχες μονάδες μέτρησης)	7
2.3.2	Συνολική παρουσίαση του τζίρου (στήλη value) για κάθε χώρα (στις αντίστοιχες μονάδες μέτρησης)	9
2.3.3	Συνολική παρουσίαση του τζίρου (στήλη value) για κάθε μέσο μεταφοράς (στις αντίστοιχες μονάδες μέτρησης)	11
2.3.4	Συνολική παρουσίαση του τζίρου (στήλη value) για κάθε μέρα της εβδομάδας (στις αντίστοιχες μονάδες μέτρησης)	13
2.3.5	Συνολική παρουσίαση του τζίρου (στήλη value) για κάθε κατηγορία εμπορεύματος (στις αντίστοιχες μονάδες μέτρησης)	15
2.3.6	Παρουσίαση των 5 μηνών με το μεγαλύτερο τζίρο, ανεξαρτήτως μέσου μεταφοράς και είδους ανακυκλώσιμων ειδών	17
2.3.7	Παρουσίαση των 5 κατηγοριών εμπορευμάτων με το μεγαλύτερο τζίρο, για κάθε χώρα	19
2.3.8	Παρουσίαση της ημέρας με το μεγαλύτερο τζίρο, για κάθε κατηγορία εμπορεύματος	22
2.4	SQL database	25
2.5	CSV files creation	29

1 Introduction

Να υλοποιηθεί **python script** που:

1. Θα συγκεντρώνει τα δεδομένα από την ιστοσελίδα: <https://www.stats.govt.nz/large-datasets/csv-files-for-download/>
2. Συγκεκριμένα, θα αντλεί δεδομένα από το "**effects-of-covid-19-ontrade-at 15-december-2021-provisional.csv1**"
3. Θα επεξεργάζεται μόνο τα απαραίτητα δεδομένα σχετικά με τα ζητούμενα γραφήματα της άσκησης.
4. Θα παρουσιάζει μενού επιλογών για τη δημιουργία τους, σε ένα γραφικό περιβάλλον.

Για την διεκπεραίωση του **Project** χρησιμοποιήθηκαν τα περιβάλλοντα:

- PyCharm, MySQL Workbench, Overleaf, Excel.
- Η στοίχιση στην αναφορά δεν έχει το ακριβές αποτέλεσμα της σύνταξης του κώδικα στο περιβάλλον σύνταξης του, γι' αυτό παραδίδονται parts αυτού και βρίσκεται αυτούσιος στο **.zip**

2 Python Coding

2.1 Libraries/Download the file from the website

Website: <https://www.stats.govt.nz/large-datasets/csv-files-for-download/>

```
import urllib
import requests
import pandas as pd
import matplotlib.pyplot as plt
import random
import calendar
from tkinter import *
import mysql.connector
import sqlalchemy

# Download the file from the website
url = "https://www.stats.govt.nz/assets/Uploads/Effects-of-COVID-19-on-trade/Effects-of-COVID-19-on-trade-At-15-December-2021-provisional/Download-data/effects-of-covid-19-on-trade-at-15-december-2021-provisional.csv"

filename = "effects-of-covid-19-on-trade.csv"

response = requests.get(url)

if response.status_code == 200:
    with open(filename, "wb") as file:
        file.write(response.content)
    print(f"The file '{filename}' has been successfully downloaded and saved.")
else:
    print("Failed to download the file.")

# Open the downloaded file
df = pd.read_csv(filename)
[...]
```

2.2 GUI creation

```
#Process the csv file
#Create the GUI

root = Tk()
root.title('Effects of COVID-19 on trade')
root.geometry("800x600")

# Styling
root.configure(bg="#40E0D0")
root.option_add('*Font', 'Helvetica 12')

# Title Label
title_label = Label(root, text="Effects of COVID-19 on Trade", font=(
    "Arial", 25), pady=20, bg="#40E0D0")
title_label.pack()

options = ["Total value per month graph"
    , "Total value per country graph"
    , "Total value per transportation mode"
    , "Total value per weekday"
    , "Total value per commodity"
    , "The 5 months with the max value"
    , "The 5 commodities with the max value per country"
    , "Day with the max value per commodity"
    ]

confirmed = StringVar()
confirmed.set("Graphs of effects of Covid-19 on trade")

[...]

dropdown_label = Label(root, text="Select an option:", font=("Arial",
    16), bg="#40E0D0")
dropdown_label.pack()

dropdown_menu = OptionMenu(root, confirmed, *options)
dropdown_menu.config(font=("Arial", 14), bg="ffffff", width=40)
dropdown_menu.pack(pady=10)

myButton = Button(root, text="Generate Graph", font=("Arial", 16))
myButton.bind("<Button-1>" , selected)
myButton.config(bg="#4caf50", fg="ffffff", width=15)
myButton.pack(pady=20)

root.mainloop()
```

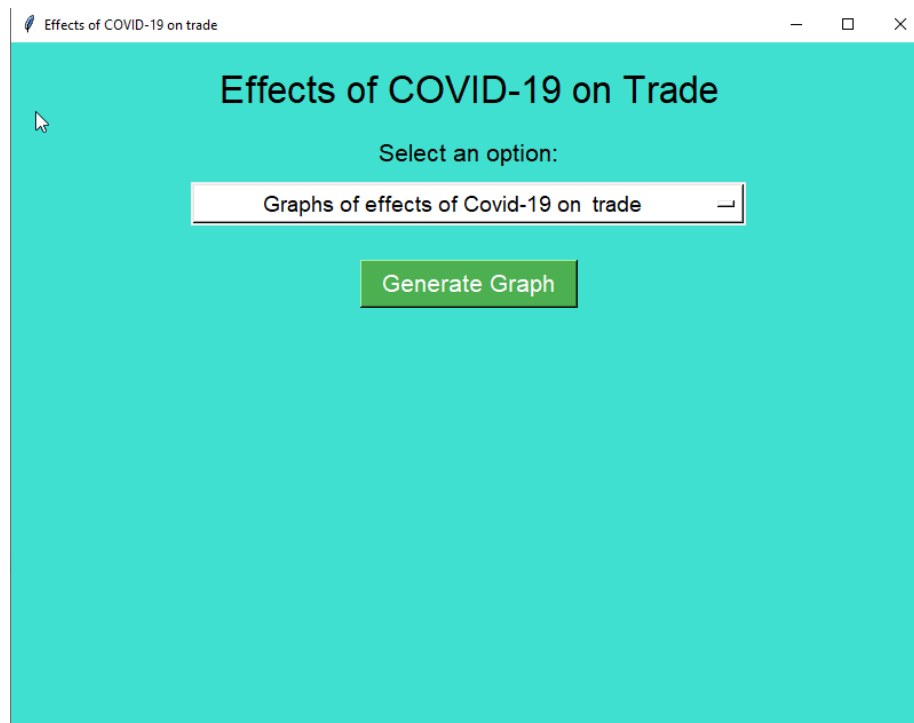


Figure 1: Interface of the Menu of Graphs

- Δημιούργησα ένα Interface το οποίο δίνει στον χρήστη την επιλογή να διαλέξει με ένα DropBox list όποιο graph από τα ζητούμενα επιθυμεί και με το button: **Generate Graph** να εμφανιστεί το figure με το αντίστοιχο graph.

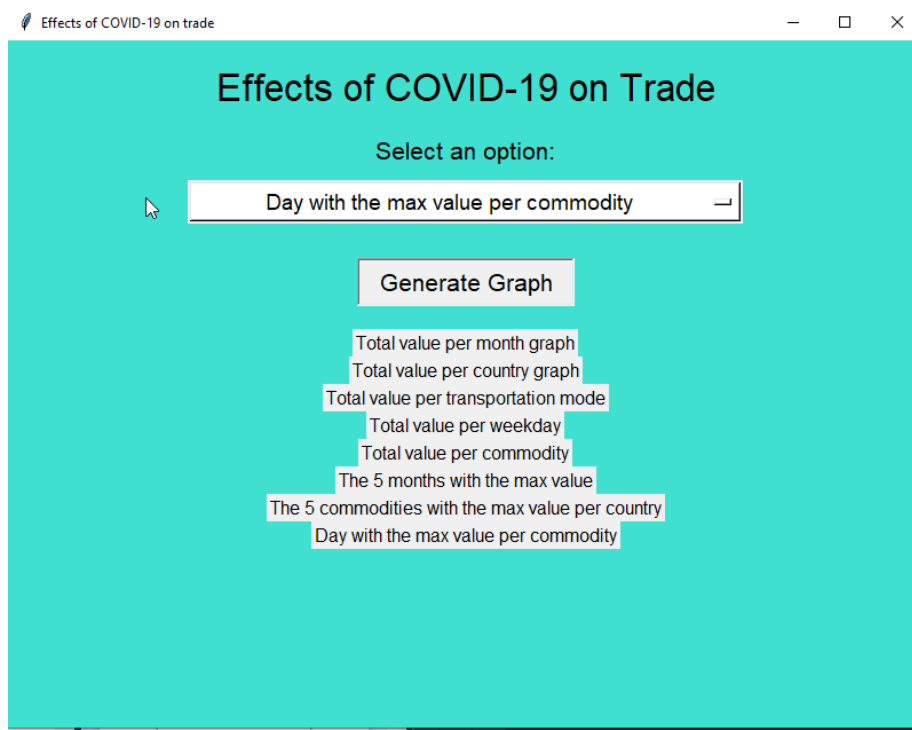


Figure 2: End of the GUI

- Στο τέλος, που έχουμε επιλέξει για εμφάνιση όλα τα graphs το interface εμφανίζει όποιο graph, με την αντίστοιχη σειρά έχει διαλέξει ο χρήστης να εμφανιστεί.

2.3 Graphs

- Για την δημιουργία των γραφημάτων και την τοποθέτηση τους μέσα στο **GUI** δημιουργήσαμε, μια συνάρτηση **def selected**, που ανάλογα με την επιστροφή της τιμής text που έχει καταχωρηθεί στην μεταβλητή confirmed, παίρνουμε και το αντίστοιχο γράφημα. Όταν το **Button: Generate Graph**, πατηθεί τότε καλείται η συνάρτηση selected και γίνεται run το αντίστοιχο part κώδικα ανάλογα με την επιλογή του χρήστη.

2.3.1 Συνολική παρουσίαση του τζιρου (στήλη value) ανά μήνα (στις αντίστοιχες μονάδες μέτρησης)

```
def selected(event):
    myLabel = Label(root, text=confirmed.get()).pack()

    if confirmed.get() == "Total value per month graph":
        # Σ
        df['Date'] = pd.to_datetime(df['Date'], format='%d/%m/%Y')
        df['Month'] = df['Date'].dt.month

        grouped_df = df.groupby(['Month', 'Measure'])['Value'].sum().
                        reset_index()

        # Get unique measure values
        measures = grouped_df['Measure'].unique()

        # Create a figure for both measures
        fig, axes = plt.subplots(2, 1, figsize=(8, 8))
        fig.suptitle("Values Per Month", fontsize=14, fontweight='bold'
                    )

        for i, measure in enumerate(measures):
            measure_df = grouped_df[grouped_df['Measure'] == measure]

            # Plot data on the corresponding subplot
            ax = axes[i]
            ax.bar(measure_df['Month'], measure_df['Value'], color=
                    tuple(random.uniform(0,
                    1) for _ in range(3)))

            ax.set_xlabel('Month', loc='center')
            ax.set_ylabel('Sum')
            ax.set_title(f"Measure: {measure}")
            ax.set_xticks(range(1, 13))
            ax.set_xticklabels(['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
                                'Jul', 'Aug', 'Sep',
                                'Oct', 'Nov', 'Dec'])

            ax.tick_params(axis='x', rotation=45)

        # Adjust the layout and spacing between subplots
        fig.tight_layout()

        # Show the figure
        plt.show()
        plt.close(fig)
```

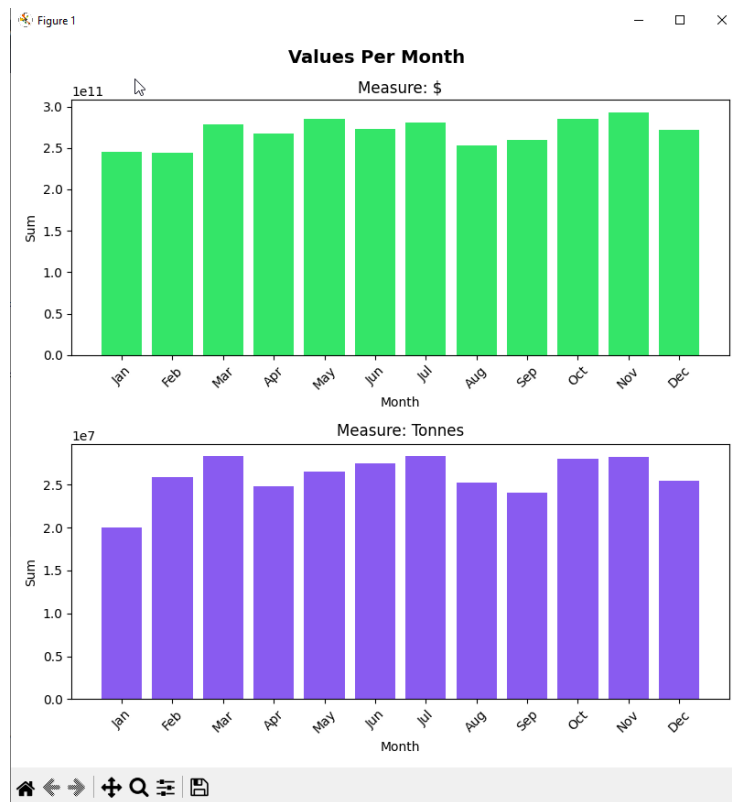


Figure 3: Values Per Month

- Αρχικά, το csv file περιλαμβάνει όλο το **Date**, οπότε από το format της ημερομηνίας εξάγουμε τον μήνα και πραγματοποιούμε την αντιστοίχία των αριθμών με τα ονόματα των μηνών.
- Ομαδοποιούμε τα δεδομένα με βάση το **Measure**, από το οποίο παίρνουμε μόνο τα unique values και το **Month**. Για αυτές τις διακριτές τιμές αθροίζουμε τις τιμές των **Values**, με την βοήθεια της συνάρτησης `sum()`.
- Τέλος, με την βοήθεια του plot δημιουργούμε το figure που θα εμφανίσει τα γραφήματα και τα παραμετροποιούμε.

2.3.2 Συνολική παρουσίαση του τζίρου (στήλη value) για κάθε χώρα (στις αντίστοιχες μονάδες μέτρησης)

```
elif confirmed.get() == "Total value per country graph":
    #Σ

    grouped_Coundf = df.groupby(['Country', 'Measure'])['Value'].
                        sum().reset_index()
    countries = grouped_Coundf['Country'].unique

    measures = grouped_Coundf['Measure'].unique()

    fig, axes = plt.subplots(2, 1, figsize=(16, 8))
    fig.suptitle("Values Per Country", fontsize=14, fontweight='
                    bold')

    for i, measure in enumerate(measures):
        measure_Coundf = grouped_Coundf[grouped_Coundf['Measure'] =
                                         = measure]

        # Plot data on the corresponding subplot
        ax = axes[i]
        ax.bar(measure_Coundf['Country'], measure_Coundf['Value'],
              color=tuple(random.uniform(0, 1) for _ in range(3)))
        ax.set_xlabel('Countries', loc='center')
        ax.set_ylabel('Sum')
        ax.set_title(f"Measure: {measure}")
        ax.tick_params(axis='x', rotation=35)

    # Adjust the layout and spacing between subplots
    fig.tight_layout()

    # Show the figure
    plt.show()
    plt.close(fig)
```

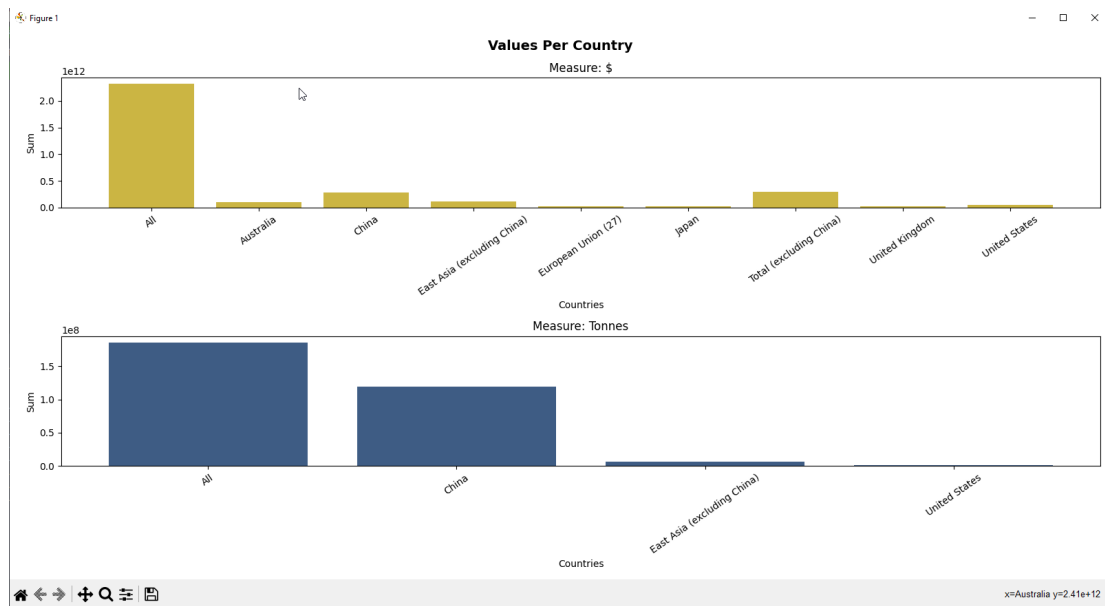


Figure 4: Values Per Country

- Ομαδοποιούμε τα δεδομένα με βάση το **Measure**, από το οποίο παίρνουμε μόνο τα unique values και το **Country**. Για αυτές τις διακριτές τιμές αθροίζουμε τις τιμές των **Values**, με την βοήθεια της συνάρτησης sum().
- Τέλος, με την βοήθεια του plot δημιουργούμε το figure που θα εμφανίσει τα γραφήματα και τα παραμετροποιούμε.

2.3.3 Συνολική παρουσίαση του τζίρου (στήλη value) για κάθε μέσο μεταφοράς (στις αντίστοιχες μονάδες μέτρησης)

```
elif confirmed.get() == "Total value per transportation mode":  
  
    # Σ  
  
    grouped_Transdf = df.groupby(['Transport_Mode', 'Measure'])['  
                                Value'].sum().reset_index()  
    transportations = grouped_Transdf['Transport_Mode'].unique()  
  
    measures = grouped_Transdf['Measure'].unique()  
  
    fig, axes = plt.subplots(2, 1, figsize=(8, 8))  
    fig.suptitle("Values Per Transport Mode", fontsize=14,  
                fontweight='bold')  
  
    for i, measure in enumerate(measures):  
        measure_Transdf = grouped_Transdf[grouped_Transdf['Measure'  
                                                             ] == measure]  
  
        # Plot data on the corresponding subplot  
        ax = axes[i]  
        ax.bar(measure_Transdf['Transport_Mode'], measure_Transdf['  
                                Value'],  
               color=tuple(random.uniform(0, 1) for _ in range(3)))  
        ax.set_xlabel('Transport')  
        ax.set_ylabel('Sum')  
        ax.set_title(f"Measure: {measure}")  
  
    # Adjust the layout and spacing between subplots  
    fig.tight_layout()  
  
    # Show the figure  
    plt.show()  
    plt.close(fig)
```

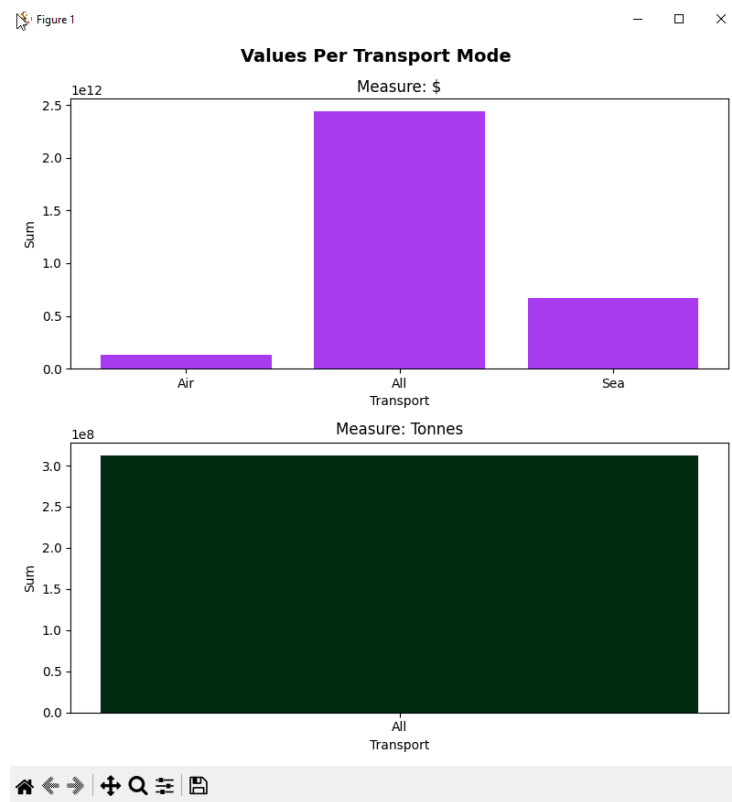


Figure 5: Values Per Transport Mode

- Ομαδοποιούμε τα δεδομένα με βάση το **Measure**, από το οποίο παίρνουμε μόνο τα unique values και το **Transport Mode**. Για αυτές τις διακριτές τιμές αθροίζουμε τις τιμές των **Values**, με την βοήθεια της συνάρτησης `sum()`.
- Τέλος, με την βοήθεια του plot δημιουργούμε το figure που θα εμφανίσει τα γραφήματα και τα παραμετροποιούμε.

2.3.4 Συνολική παρουσίαση του τζίρου (στήλη value) για κάθε μέρα της εβδομάδας (στις αντίστοιχες μονάδες μέτρησης)

```
elif confirmed.get() == "Total value per weekday":  
  
    # Σ  
  
    grouped_WeekDaydf = df.groupby(['Weekday', 'Measure'])['Value']  
                           .sum().reset_index()  
    weekday = grouped_WeekDaydf['Weekday'].unique  
  
    measures = grouped_WeekDaydf['Measure'].unique()  
  
    fig, axes = plt.subplots(2, 1, figsize=(8, 8))  
    fig.suptitle("Values Per Weekday", fontsize=14, fontweight='  
                    bold')  
  
    for i, measure in enumerate(measures):  
        measure_WeekDaydf = grouped_WeekDaydf[grouped_WeekDaydf['  
                                                    Measure'] == measure]  
  
        # Plot data on the corresponding subplot  
        ax = axes[i]  
        ax.bar(measure_WeekDaydf['Weekday'], measure_WeekDaydf['  
                                                    Value'],  
               color=tuple(random.uniform(0, 1) for _ in range(3)))  
        ax.set_xlabel('Weekday')  
        ax.set_ylabel('Sum')  
        ax.set_title(f"Measure: {measure}")  
        ax.tick_params(axis='x', rotation=35)  
  
    # Adjust the layout and spacing between subplots  
    fig.tight_layout()  
  
    # Show the figure  
    plt.show()  
    plt.close(fig)
```

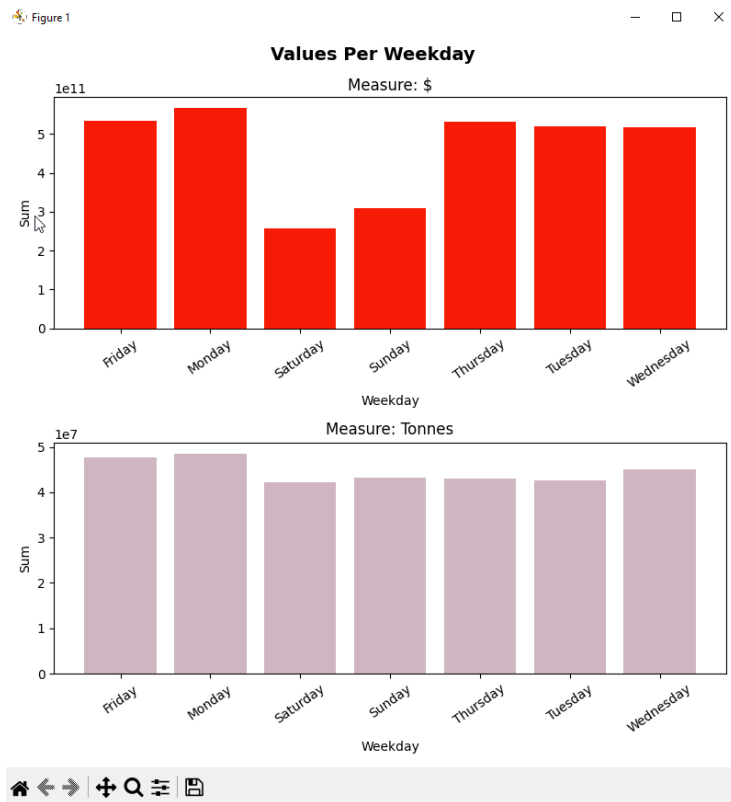


Figure 6: Values Per Weekday

- Ομαδοποιούμε τα δεδομένα με βάση το **Measure**, από το οποίο παίρνουμε μόνο τα unique values και το **Weekday**. Για αυτές τις διακριτές τιμές αθροίζουμε τις τιμές των **Values**, με την βοήθεια της συνάρτησης `sum()`.
- Τέλος, με την βοήθεια του `plot` δημιουργούμε το `figure` που θα εμφανίσει τα γραφήματα και τα παραμετροποιούμε.

2.3.5 Συνολική παρουσίαση του τζιρου (στήλη value) για κάθε κατηγορία εμπορεύματος (στις αντίστοιχες μονάδες μέτρησης)

```
elif confirmed.get() == "Total value per commodity":  
  
    # Σ  
  
    grouped_Commdf = df.groupby(['Commodity', 'Measure'])['Value'].  
                        sum().reset_index()  
    commodity = grouped_Commdf['Commodity'].unique()  
  
    num_commodities = len(commodity)  
  
    measures = grouped_Commdf['Measure'].unique()  
  
    fig, axes = plt.subplots(2, 1, figsize=(8, 8))  
    fig.suptitle("Values Per Commodity", fontsize=14, fontweight='  
                    bold')  
  
    for i, measure in enumerate(measures):  
        measure_Commdf = grouped_Commdf[grouped_Commdf['Measure'] =  
                                         = measure]  
  
        # Plot data on the corresponding subplot  
        ax = axes[i]  
        ax.bar(measure_Commdf['Commodity'], measure_Commdf['Value']  
               ,  
               color=tuple(random.uniform(0, 1) for _ in range(3)))  
        ax.set_xlabel('Commodity')  
        ax.set_ylabel('Sum')  
        ax.set_title(f"Measure: {measure}")  
        ax.set_xticks(range(num_commodities))  
        ax.set_xticklabels(commodity, rotation=45, ha='right')  
  
    # Adjust the layout and spacing between subplots  
    fig.tight_layout()  
  
    # Show the figure  
    plt.show()  
    plt.close(fig)
```

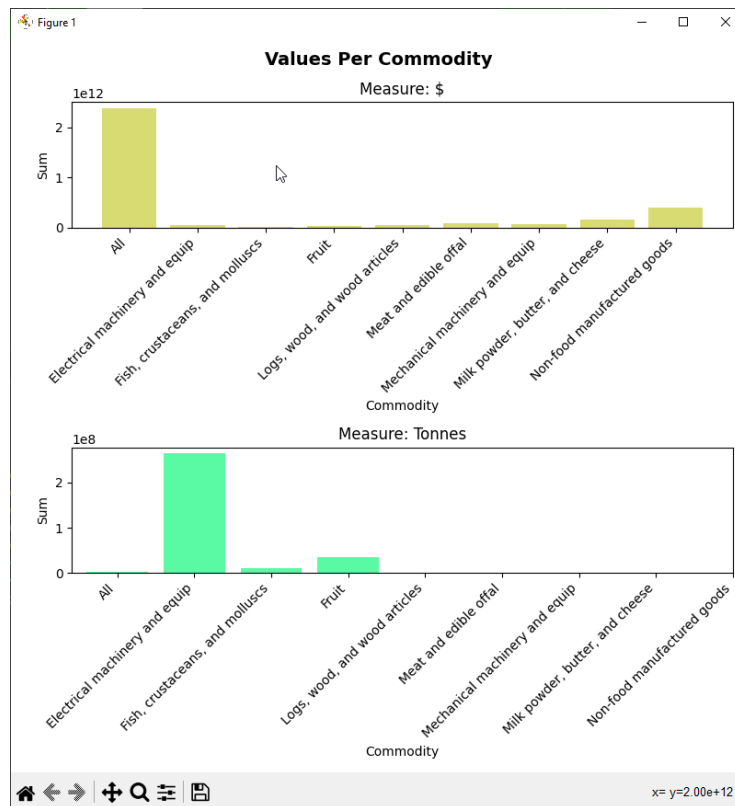


Figure 7: Values Per Commodity

- Ομαδοποιούμε τα δεδομένα με βάση το **Measure**, από το οποίο παίρνουμε μόνο τα unique values και το **Commodity**. Για αυτές τις διακριτές τιμές αθροίζουμε τις τιμές των **Values**, με την βοήθεια της συνάρτησης `sum()`.
- Τέλος, με την βοήθεια του plot δημιουργούμε το figure που θα εμφανίσει τα γραφήματα και τα παραμετροποιούμε.

2.3.6 Παρουσίαση των 5 μηνών με το μεγαλύτερο τζίρο, ανεξαρτήτως μέσου μεταφοράς και είδους ανακυκλώσιμων ειδών

```
elif confirmed.get() == "The 5 months with the max value":

    # Π          5          ,

    df['Date'] = pd.to_datetime(df['Date'], format='%d/%m/%Y')
    df['Month'] = df['Date'].dt.month

    grouped_df = df.groupby(['Month', 'Measure'])['Value'].sum().
                    reset_index()

    # Get unique measure values
    measures = grouped_df['Measure'].unique()

    # Create a figure for both measures
    fig, axes = plt.subplots(2, 1, figsize=(8, 8))
    fig.suptitle("The 5 months with the maximum value", fontsize=14
                  , fontweight='bold')

    for i, measure in enumerate(measures):
        measure_df = grouped_df[grouped_df['Measure'] == measure]

        top5_months = measure_df.nlargest(5, 'Value')

        # Plot data on the corresponding subplot
        ax = axes[i]
        ax.plot(top5_months['Month'], top5_months['Value'], marker=
                'o',
                color=tuple(random.uniform(0, 1) for _ in range(3)))
        ax.set_xlabel('Month', loc='center')
        ax.set_ylabel('Sum')
        ax.set_title(f"Measure: {measure}")
        month_names = [calendar.month_name[month] for month in
                        top5_months['Month']]
        ax.set_xticks(top5_months['Month'])
        ax.set_xticklabels(month_names, rotation=45, ha='right')

    # Adjust the layout and spacing between subplots
    fig.tight_layout()

    # Show the figure
    plt.show()
    plt.close(fig)
```

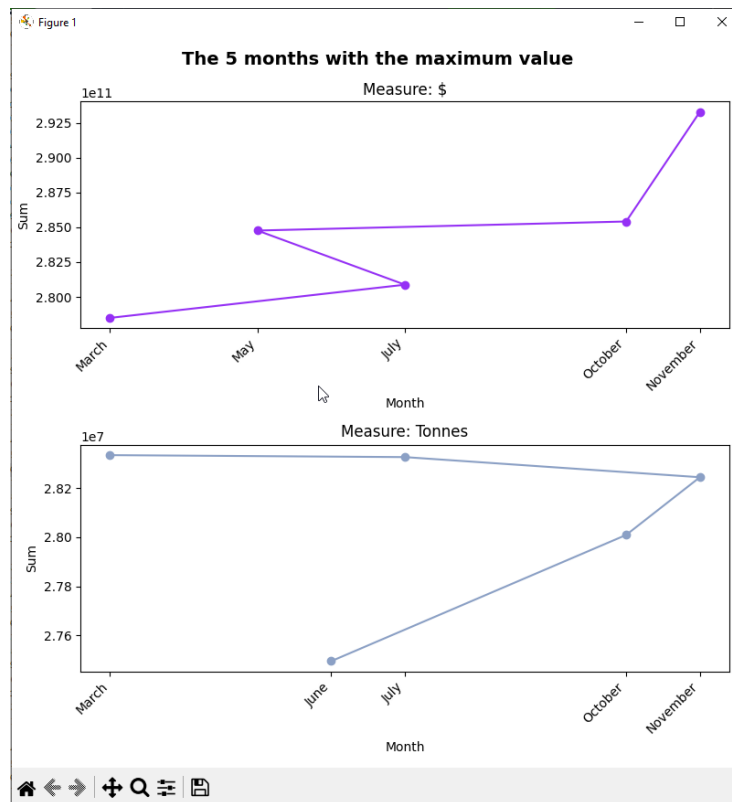


Figure 8: Top 5 months with the maximum value

- Ομαδοποιούμε τα δεδομένα με βάση το **Measure**, από το οποίο παίρνουμε μόνο τα unique values και το **Month**, από το οποίο με την βοήθεια της συνάρτησης `nlargest()` παίρνουμε τις μεγαλύτερες τιμές του αθροίσματος των Values.
- Για αυτές τις διακριτές τιμές αθροίζουμε τις τιμές των **Values**, με την βοήθεια της συνάρτησης `sum()`.
- Τέλος, με την βοήθεια του `plot` δημιουργούμε το figure που θα εμφανίσει τα γραφήματα και τα παραμετροποιούμε.

2.3.7 Παρουσίαση των 5 κατηγοριών εμπορευμάτων με το μεγαλύτερο τζίρο, για κάθε χώρα

```
elif confirmed.get() == "The 5 commodities with the max value
                                per country":

    # Group by 'Country' and 'Commodity' and calculate the sum of '
    Value'
    grouped_maxCommdf = df.groupby(['Country', 'Commodity'])['Value
                                '].sum().reset_index()

    # Get unique country values
    countries = grouped_maxCommdf['Country'].unique()

    # Calculate the number of countries
    num_countries = len(countries)

    # Define the number of subplots per row and column
    subplots_per_row = 2
    subplots_per_column = (num_countries + 1) // subplots_per_row

    # Calculate the total figure size based on the number of
    subplots
    fig_width = 15 * subplots_per_row
    fig_height = 8 * subplots_per_column

    # Create a figure for all countries
    fig_all, axes_all = plt.subplots(subplots_per_column,
                                subplots_per_row, figsize=(
                                fig_width, fig_height))

    fig_all.suptitle('Top 5 Commodities by Country', fontsize=14,
                                fontweight='bold')

    # Iterate over the countries
    for i, country in enumerate(countries):
        country_df = grouped_maxCommdf[grouped_maxCommdf['Country']
                                == country]

        # Get the top 5 commodities with the maximum values
        top_commodities = country_df.sort_values('Value', ascending
                                =False).head(5)

        # Calculate the subplot index
        subplot_row = i // subplots_per_row
        subplot_col = i % subplots_per_row

        # Plot data on the current country's subplot in the main
        figure
        ax = axes_all[subplot_row, subplot_col]
        ax.bar(range(len(top_commodities)), top_commodities['Value'
                                ],
                                color=tuple(random.uniform(0, 1) for _ in range(3)))
        ax.set_title(country)

        # Set the x-ticks and labels
        ax.set_xticks(range(len(top_commodities)))
```

```

        ax.set_xticklabels(top_commodities['Commodity'], rotation=
                           15, ha='right')

# Remove any unused subplots in the figure for all countries
for i in range(num_countries, subplots_per_row *
               subplots_per_column):
    subplot_row = i // subplots_per_row
    subplot_col = i % subplots_per_row
    fig_all.delaxes(axes_all[subplot_row, subplot_col])

# Adjust the spacing between subplots in the figure for all countries
plt.subplots_adjust(wspace=0.5, hspace=1.5)

# Show the figure for all countries
plt.show()
plt.close()

```

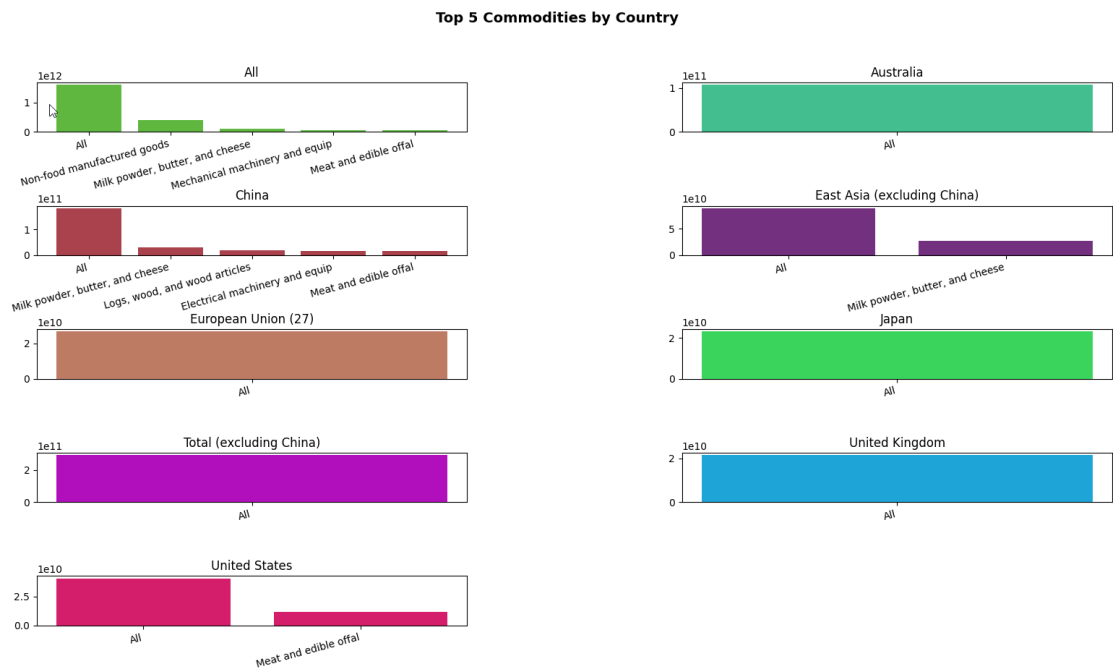


Figure 9: Graph: Top 5 commodities per country with the maximum value

- Ομαδοποιούμε τα δεδομένα με βάση το **Country**, από το οποίο παίρνουμε μόνο τα unique values και το **Commodity**, από το οποίο με την βοήθεια της ταξινόμησης και της συνάρτησης head(5) παίρνουμε τις 5 μεγαλύτερες τιμές του αθροίσματος των Values.
- Για αυτές τις διακριτές τιμές αθροίζουμε τις τιμές των **Values**, με την βοήθεια της συνάρτησης sum().
- Τέλος, με την βοήθεια του plot δημιουργούμε το figure που θα εμφανίσει τα γραφήματα και τα παραμετροποιούμε.

2.3.8 Παρουσίαση της ημέρας με το μεγαλύτερο τζίρο, για κάθε κατηγορία εμπορεύματος

```
elif confirmed.get() == "Day with the max value per commodity":
    # Π
    # Group by 'Country' and calculate the sum of 'Value'
    grouped_maxWeekdf = df.groupby(['Weekday', 'Commodity'])['Value']
    grouped_maxWeekdf = grouped_maxWeekdf.sum().reset_index()

    # Get unique day values
    weekday = grouped_maxWeekdf['Weekday'].unique()

    num_weekday = len(weekday)

    # Define the number of subplots per row and column
    subplots_per_row = 2
    subplots_per_column = (num_weekday + 1) // subplots_per_row

    # Calculate the total figure size based on the number of subplots
    fig_width = 15 * subplots_per_row
    fig_height = 8 * subplots_per_column

    # Create a figure for all days
    fig_all, axes_all = plt.subplots(subplots_per_column,
                                     subplots_per_row, figsize=(
                                         fig_width, fig_height))

    fig_all.suptitle('Day with the maximum value for each commodity',
                    fontsize=14, fontweight='bold')

    # Iterate over the week
    for i, day in enumerate(weekday):
        weekday_df = grouped_maxWeekdf[grouped_maxWeekdf['Weekday']
                                       == day]

        # Get the top 5 commodities with the maximum values
        top_commodities = weekday_df.sort_values('Value', ascending
                                                =False)

        # Calculate the subplot index
        subplot_row = i // subplots_per_row
        subplot_col = i % subplots_per_row

        # Plot data on the current subplot in the main figure
        ax = axes_all[subplot_row, subplot_col]
        ax.bar(range(len(top_commodities)), top_commodities['Value'],
              color=tuple(random.uniform(0, 1) for _ in range(3)))
        ax.set_title(day)

        # Set the x-ticks and labels
        ax.set_xticks(range(len(top_commodities)))
        ax.set_xticklabels(top_commodities['Commodity'], rotation=
                           15, ha='right')
```

```

# Remove any unused subplots in the figure for all days
for i in range(num_weekday, subplots_per_row *
                subplots_per_column):
    subplot_row = i // subplots_per_row
    subplot_col = i % subplots_per_row
    fig_all.delaxes(axes_all[subplot_row, subplot_col])

# Adjust the spacing between subplots in the figure for all
# days
plt.subplots_adjust(wspace=0.5, hspace=1.5)

# Show the figure for all days
plt.show()
plt.close()

```

Day with the maximum value for each commodity

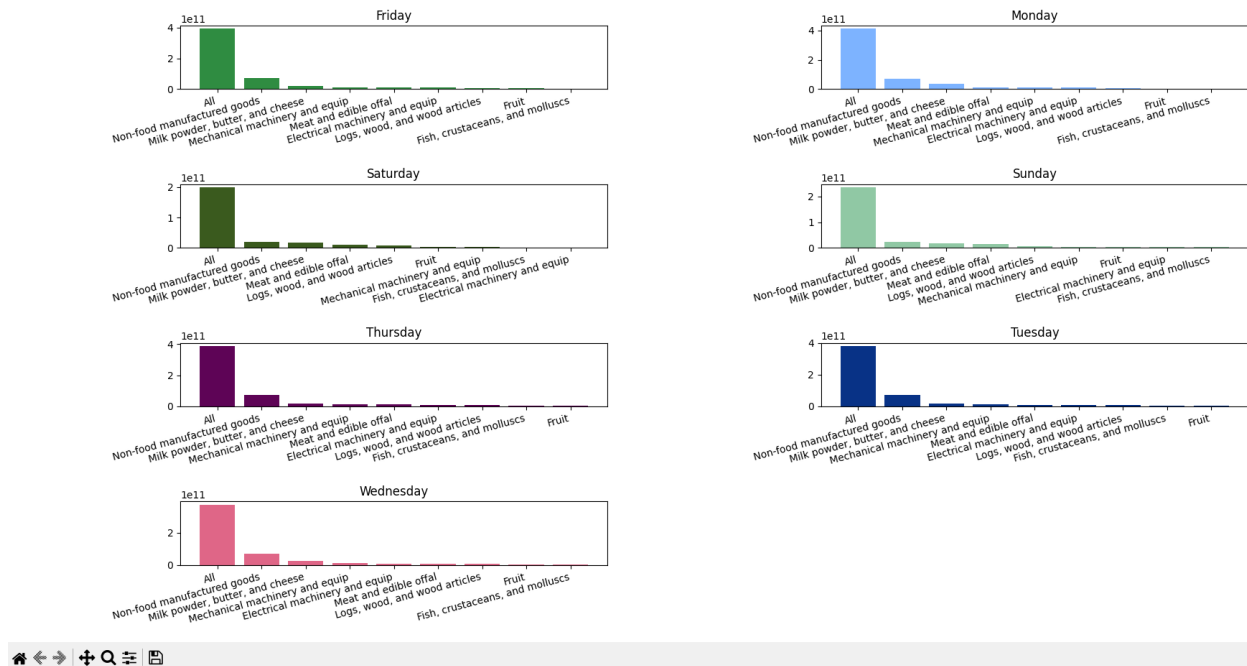


Figure 10: Top 5 commodities per country with the maximum value

- Ομαδοποιούμε τα δεδομένα με βάση το **Weekday**, από το οποίο παίρνουμε μόνο τα unique values και το **Commodity**, από το οποίο με την βοήθεια παίρνουμε τις μεγαλύτερες τιμές του αθροίσματος των Values για κάθε κατηγορία εμπορεύματος.
- Για αυτές τις διακριτές τιμές αθροίζουμε τις τιμές των **Values**, με την βοήθεια της συνάρτησης `sum()`.
- Τέλος, με την βοήθεια του `plot` δημιουργούμε το figure που θα εμφανίσει τα γραφήματα και τα παραμετροποιούμε.

2.4 SQL database

```
#Connect with my database
mydb = mysql.connector.connect (
    host = "localhost",
    user = "root",
    password = "soulele3059@",
    port= "3306" ,
    database="EffectsOfCovid"
)

mycursor = mydb.cursor()
mycursor.execute("CREATE TABLE IF NOT EXISTS total_value_per_month (
    valueMon DOUBLE NOT NULL, "
    month VARCHAR(50) NOT NULL, "
    measure VARCHAR(24) NOT NULL)")

df['Date'] = pd.to_datetime(df['Date'], format='%d/%m/%Y')
df['Month'] = df['Date'].dt.month
grouped_df = df.groupby(['Month', 'Measure'])['Value'].sum().
    reset_index()
# Convert DataFrame values to a list of tuples
values = grouped_df[['Value', 'Month', 'Measure']].to_records(index=
    False).tolist()

mycursor.executemany("INSERT INTO total_value_per_month (valueMon,
    month, measure) VALUES (%s, %s, %s)
    ", values)

mycursor.execute("CREATE TABLE IF NOT EXISTS total_value_per_country (
    valueCoun DOUBLE NOT NULL, "
    country VARCHAR(50) NOT NULL ,
    measure VARCHAR(24) NOT NULL)")

groupedCoun_df = df.groupby(['Country', 'Measure'])['Value'].sum().
    reset_index()
values = groupedCoun_df[['Value', 'Country', 'Measure']].to_records(
    index=False).tolist()
mycursor.executemany("INSERT INTO total_value_per_country (valueCoun,
    country, measure) VALUES (%s, %s, %s)
    ", values)

mycursor.execute("CREATE TABLE IF NOT EXISTS total_value_per_transport
    (
    valueTrans DOUBLE NOT NULL, "
    transportation VARCHAR(50) NOT NULL,"
    measure VARCHAR(24) NOT NULL)")
groupedTrans_df = df.groupby(['Transport_Mode', 'Measure'])['Value'].
    sum().reset_index()
values = groupedTrans_df[['Value', 'Transport_Mode', 'Measure']].
    to_records(index=False).tolist()
mycursor.executemany("INSERT INTO total_value_per_transport (valueTrans
    , transportation, measure) VALUES (
    %s, %s, %s) ", values)
```

```

mycursor.execute("CREATE TABLE IF NOT EXISTS total_value_per_day ("
    "valueWeek DOUBLE NOT NULL, "
    "weekday VARCHAR(50) NOT NULL,"
    "measure VARCHAR(24) NOT NULL)")
groupedWeek_df = df.groupby(['Weekday', 'Measure'])['Value'].sum().
    reset_index()
values = groupedWeek_df[['Value', 'Weekday', 'Measure']].to_records(
    index=False).tolist()
mycursor.executemany("INSERT INTO total_value_per_day (valueWeek,
    weekday, measure) VALUES (%s, %s, %s) ", values)

mycursor.execute("CREATE TABLE IF NOT EXISTS total_value_per_commodity
    ("
    "valueCom DOUBLE NOT NULL, "
    "commodity VARCHAR(50) NOT NULL , "
    "measure VARCHAR(24) NOT NULL)")
groupedComm_df = df.groupby(['Commodity', 'Measure'])['Value'].sum().
    reset_index()
values = groupedComm_df[['Value', 'Commodity', 'Measure']].to_records(
    index=False).tolist()
mycursor.executemany("INSERT INTO total_value_per_commodity (valueCom,
    commodity, measure) VALUES (%s, %s, %s) ", values)

mycursor.execute("CREATE TABLE IF NOT EXISTS
    top_five_months_per_maxvalue("
    "valueMax DOUBLE NOT NULL, "
    "month VARCHAR(50) NOT NULL, "
    "measure VARCHAR(24) NOT NULL)")
values = grouped_df[['Value', 'Month', 'Measure']].to_records(index=
    False).tolist()
mycursor.executemany("INSERT INTO top_five_months_per_maxvalue (
    valueMax, month, measure) VALUES (%s, %s, %s) ", values)

mycursor.execute("CREATE TABLE IF NOT EXISTS
    top_five_commos_per_maxvalue ("
    "valueMaxCom DOUBLE NOT NULL, "
    "commodity VARCHAR(50) NOT NULL, "
    "country VARCHAR(50) NOT NULL)")
grouped_maxComm_df = df.groupby(['Commodity', 'Country'])['Value'].sum().
    reset_index()
values = grouped_maxComm_df[['Value', 'Commodity', 'Country']].
    to_records(index=False).tolist()
mycursor.executemany("INSERT INTO top_five_commos_per_maxvalue (
    valueMaxCom, commodity, country)
    VALUES (%s, %s, %s)", values)

mycursor.execute("CREATE TABLE IF NOT EXISTS top_day_per_maxvalue ("
    "valueMaxDay DOUBLE NOT NULL, "
    "weekday VARCHAR(50) NOT NULL, "
    "commodity VARCHAR(50) NOT NULL)")

```

```

grouped_maxWeekdf = df.groupby(['Weekday' , 'Commodity'])['Value'].sum
                        ().reset_index()
values = grouped_maxWeekdf[['Value', 'Weekday', 'Commodity']].
                        to_records(index=False).tolist()
mycursor.executemany("INSERT INTO top_day_per_maxvalue (valueMaxDay,
                        weekday, commodity) VALUES (%s, %s,
                        %s)", values)

encoded_password = urllib.parse.quote_plus("soulele3059@")

# Create the engine
engine = sqlalchemy.create_engine(f'mysql+mysqlconnector://root:{
                        encoded_password}@localhost:3306/
                        EffectsOfCovid')

```

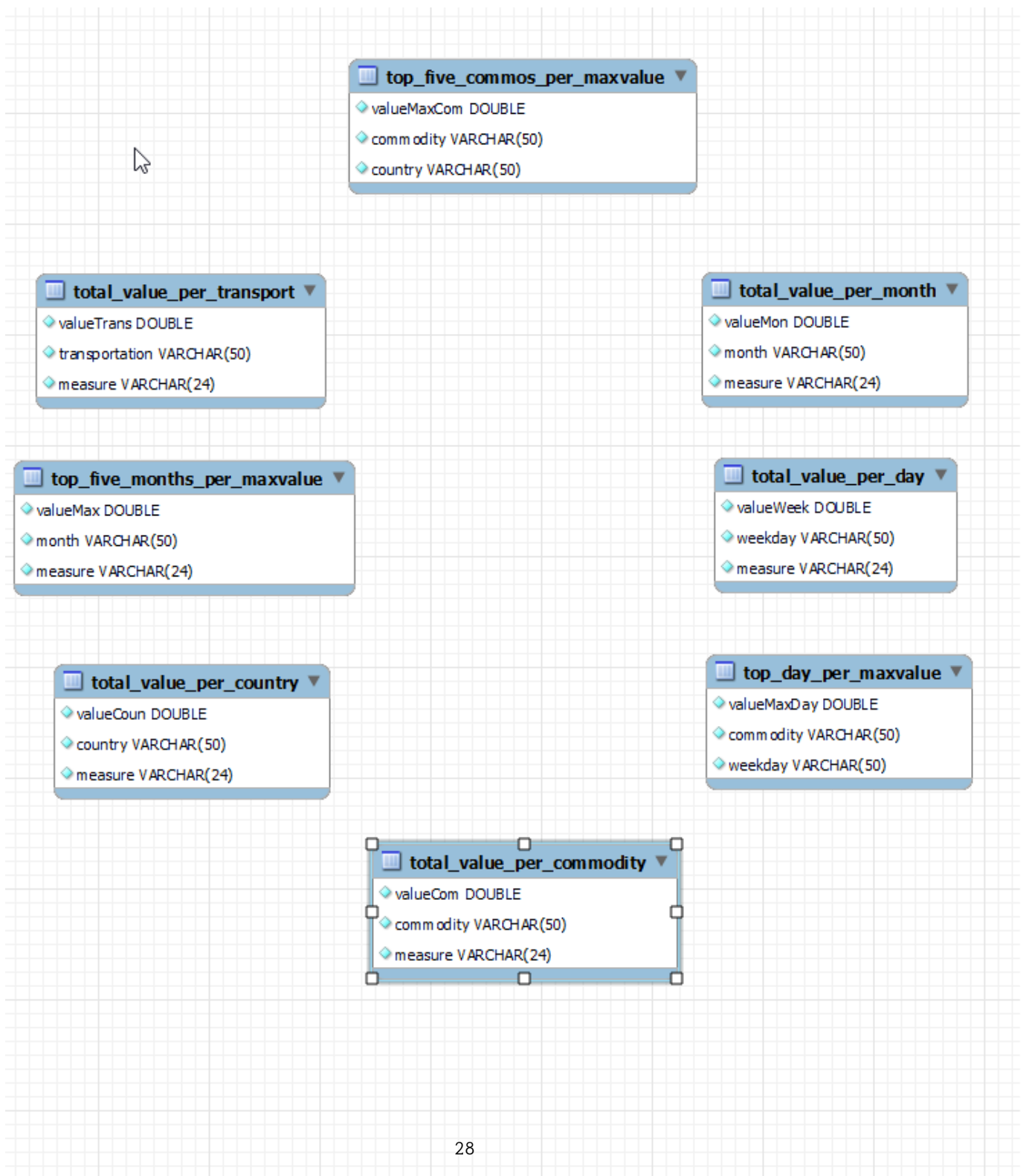


Figure 11: Schema of the Database: Effects of COVID-19

2.5 CSV files creation

```
# Execute SQL queries and save results to CSV files
sql_query1 = "SELECT * FROM total_value_per_month"
sql_query2 = "SELECT * FROM total_value_per_country"
sql_query3 = "SELECT * FROM total_value_per_transport"
sql_query4 = "SELECT * FROM total_value_per_day"
sql_query5 = "SELECT * FROM total_value_per_commodity"
sql_query6 = "SELECT * FROM top_five_months_per_maxvalue"
sql_query7 = "SELECT * FROM top_five_commos_per_maxvalue"
sql_query8 = "SELECT * FROM top_day_per_maxvalue"

sql_query1_df = pd.read_sql_query(sql_query1, engine)
sql_query2_df = pd.read_sql_query(sql_query2, engine)
sql_query3_df = pd.read_sql_query(sql_query3, engine)
sql_query4_df = pd.read_sql_query(sql_query4, engine)
sql_query5_df = pd.read_sql_query(sql_query5, engine)
sql_query6_df = pd.read_sql_query(sql_query6, engine)
sql_query7_df = pd.read_sql_query(sql_query7, engine)
sql_query8_df = pd.read_sql_query(sql_query8, engine)

# Export DataFrames to CSV files
sql_query1_df.to_csv(r"C:\Python\ProjectPython\total_value_per_month.
                    csv", index=False)
sql_query2_df.to_csv(r"C:\Python\ProjectPython\total_value_per_country.
                    csv", index=False)
sql_query3_df.to_csv(r"C:\Python\ProjectPython\total_value_per_transport.csv",
                    index=False)
sql_query4_df.to_csv(r"C:\Python\ProjectPython\total_value_per_day.csv",
                    index=False)
sql_query5_df.to_csv(r"C:\Python\ProjectPython\total_value_per_commodity.csv",
                    index=False)
sql_query6_df.to_csv(r"C:\Python\ProjectPython\top_five_months_per_maxvalue.csv",
                    index=False)
sql_query7_df.to_csv(r"C:\Python\ProjectPython\top_five_commos_per_maxvalue.csv",
                    index=False)
sql_query8_df.to_csv(r"C:\Python\ProjectPython\top_day_per_maxvalue.csv",
                    index=False)

mydb.commit()
mycursor.close()
mydb.close()
```